

Avances del Proyecto – Sistema de IA Anticipatoria para el STC Metro

Alumno: Alan Bellon García
Asesor: M. en Fil. C. Enrique Francisco Soto Astorga
Fecha de este reporte: 2026-09-13

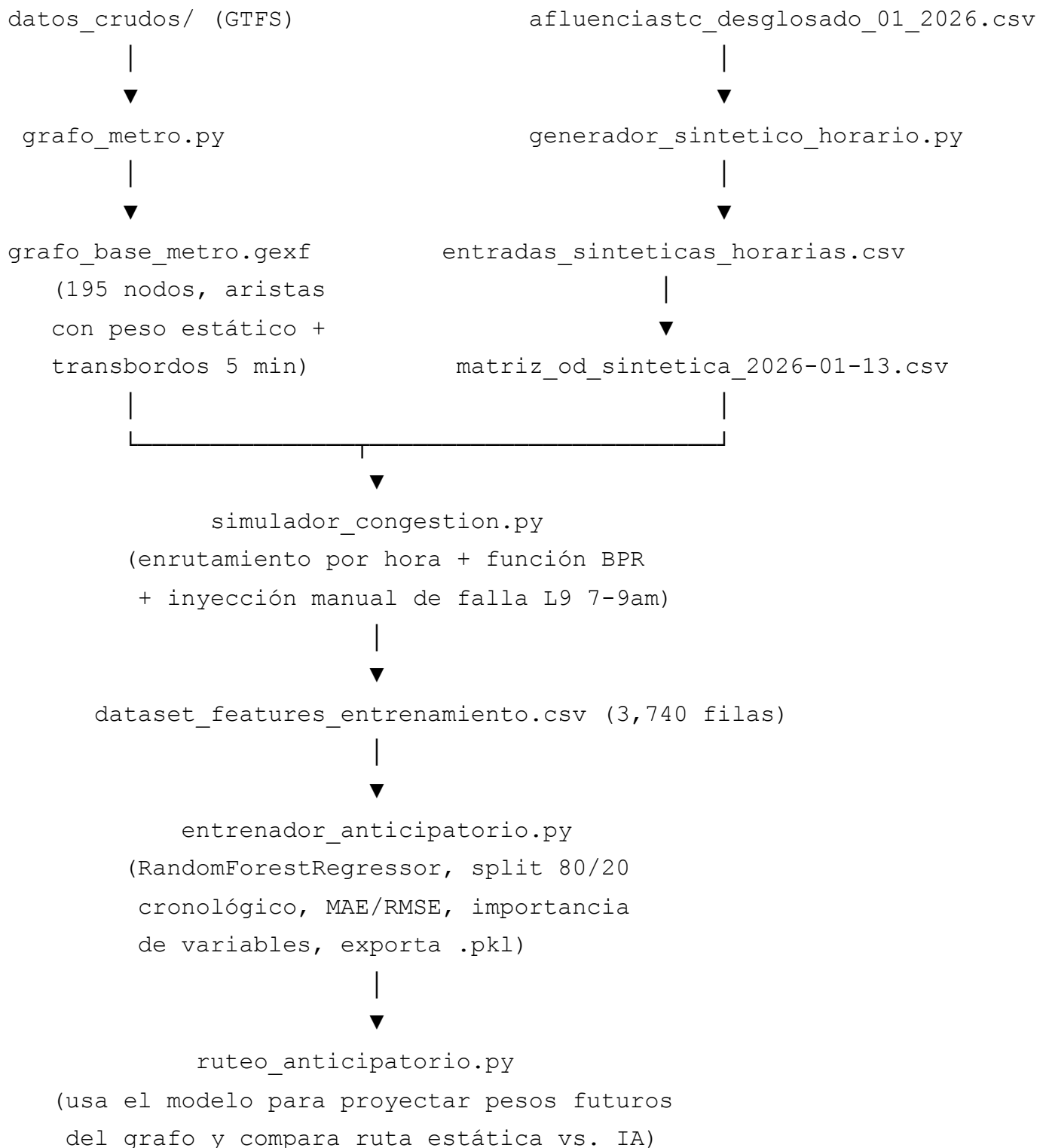
Este documento resume el estado técnico y metodológico del prototipo descrito en el anexo de titulación (*"Modelado y prototipado de un sistema de Inteligencia Artificial Anticipatoria para la estimación de estados de congestión en el STC Metro de la Ciudad de México"*), con base en el código y los datos actualmente presentes en `00_Programas/tesis_metro_ai/`.

1. Mapeo contra los objetivos de la tesis

Objetivo secundario (Anexo)	Estado	Evidencia en código
Generar dataset sintético de afluencia/disrupciones	✅ Completo (versión inicial, un solo día simulado)	<code>generador_sintetico_horario.py</code> , <code>datos_procesados/*.csv</code>
Representar la red como grafo con pesos	✅ Completo	<code>grafo_metro.py</code> → <code>grafo_base_metro.gexf</code>
Modelo de estimación a horizonte corto (10–60 min)	⚠️ Parcial (horizonte discreto de 1 hora, no continuo 10–60 min)	<code>entrenador_anticipatorio.py</code> → <code>modelos/modelo_anticipatorio_rf.pkl</code>
Algoritmo de ruteo que integre la métrica predictiva	✅ Prueba de concepto funcional	<code>ruteo_anticipatorio.py</code>
Integración estimación + ruteo en prototipo funcional	✅ Demostrado en un caso (Pantitlán→Auditorio, 7:00)	<code>ruteo_anticipatorio.py</code>
Explicabilidad de las recomendaciones	✅ Iniciado (importancia de variables)	<code>entrenador_anticipatorio.py</code> → <code>importancia_variables.png</code>
Sistema reactivo de comparación (índice 5.5)	✅ Implementado como baseline dentro del mismo script	<code>ruteo_anticipatorio.py</code> (ruta estática vs. ruta IA)

En términos del índice tentativo, el proyecto ha cubierto el **capítulo 4** completo (modelado y generación de datos) y tiene un **primer corte funcional del capítulo 5** (diseño del prototipo: arquitectura, modelo predictivo, ruteo, baseline reactivo). Falta evaluación formal de KPIs (5.4), profundizar explicabilidad (6.5) y ejecutar comparativas sistemáticas (6.4) sobre múltiples escenarios, no solo el caso de prueba único.

2. Pipeline técnico actual (orden de ejecución)



2.1 `grafo_metro.py` – Modelado topológico

- Filtra GTFS masivo del Valle de México por `agency_id == "METRO"` para aislar rutas, viajes y `stop_times`.
- Construye grafo no dirigido $G(V, E, W)$ con **195 nodos** (andenes/estaciones).
- Peso estático de cada arista = promedio de `departure_sec` → `arrival_sec` entre paradas consecutivas del mismo `trip_id`, filtrando deltas ilógicos (`<0` o `>1800s`).
- Agrega aristas de **transbordo peatonal** (5 min fijos) entre andenes que comparten `stop_name`, habilitando cambios de línea en el grafo.
- Exporta a `grafo_base_metro.gexf` (formato estándar, reutilizable por NetworkX).

Limitación notable: el penalti de transbordo es una constante (5 min) igual para todas las estaciones de correspondencia, sin distinguir estaciones con transbordos largos conocidos (p. ej. Ciudad Azteca, La Raza, Tacubaya).

2.2 generador_sintetico_horario.py – Motor de demanda

- Limpia encoding (latin1→utf-8) de estación/línea provenientes del portal de datos abiertos.
- Clasifica estaciones en 3 perfiles espaciotemporales (`origen`, `destino`, `mixto`) mediante listas fijas (hardcoded) de 10 estaciones por extremo.
- Desagrega afluencia diaria a horaria multiplicando por curvas bimodales (`peso_origen`, `peso_destino`, `peso_mixto`) calibradas manualmente para 24 horas.
- **Matriz Origen-Destino sintética** vía modelo gravitacional simplificado: calcula `atractividad_destino` según hora y perfil, normaliza por estación de origen y reparte las entradas reales observadas como viajes probables a cada destino.
- Output: `entradas_sinteticas_horarias.csv` y `matriz_od_sintetica_2026-01-13.csv`
(actualmente **un solo día simulado**, `2026-01-13`).

2.3 simulador_congestion.py – Motor de estrés dinámico

- Carga el grafo base y la matriz O-D del día simulado.
- Por cada hora de operación (5-23h), copia el grafo, enruta cada flujo O-D con `nx.shortest_path` (ponderado por `tiempo_minutos`) y acumula `carga_pasajeros` por arista.
- Aplica **función BPR adaptada** para convertir carga en tiempo congestionado:

$$T_c = T_b \times (1 + \alpha \cdot (V/C)^\beta), \alpha=0.15, \beta=4, \text{ tope en } 4 \times T_b$$

con `CAPACIDAD_PROMEDIO_TRAMO_HORA = 35000` (constante única para toda la red).

- **Inyección de interrupción determinista:** entre 7–9h, cualquier arista cuyo nodo contenga el string `"B_0200L9"` recibe +40,000 pasajeros fantasma para simular una falla de Línea 9. Esto es un evento *hardcodeado*, no estocástico.
- Construye ventanas temporales ($t-1, t, t+1$) de `congestibilidad` por tramo y exporta `dataset_features_entrenamiento.csv` (3,740 registros) como tabla de entrenamiento.

2.4 `entrenador_anticipatorio.py` – Modelo predictivo

- Ordena cronológicamente y separa train/test 80/20 **sin aleatorizar** (evita fuga de información temporal).
- Features: `hora`, `tiempo_ideal`, `congestibilidad_t`, `congestibilidad_t_minus_1`.
Target: `target_congestibilidad_t_plus_1`.
- `RandomForestRegressor` (150 árboles, profundidad 10) evaluado con MAE/RMSE.
- Genera gráfica de importancia de variables (`importancia_variables.png`) como primer paso de explicabilidad.
- Exporta modelo a `modelos/modelo_anticipatorio_rf.pkl`.

2.5 `ruteo_anticipatorio.py` – Integración estimación + ruteo

- Carga grafo base + modelo entrenado + dataset de contexto (usado como proxy de "sensores en tiempo real").
- `crear_grafo_futuro(hora)` : para cada arista con datos en esa hora, predice el retraso futuro y lo suma al tiempo ideal, generando un grafo proyectado.
- Compara ruta estática (Dijkstra sobre tiempo ideal) vs. ruta anticipatoria (Dijkstra sobre pesos proyectados por IA), evaluando ambas contra el tráfico real proyectado.
- Caso de prueba demostrado: Pantitlán → Auditorio, 7:00 AM.

3. Limitaciones metodológicas actuales (a atender antes de tesis final)

1. **Un solo día de simulación** (`2026-01-13`): el dataset de entrenamiento no captura variabilidad día-a-día (fin de semana vs. entre semana, quincena, clima), lo cual restringe la capacidad de generalización del modelo y hace que el split 80/20 temporal, en la práctica, separe *horas* dentro del mismo día, no días distintos.
2. **Evento de interrupción hardcodeado:** la falla de Línea 9 es determinista (mismas horas, mismo tramo, misma magnitud siempre), no un proceso estocástico. El modelo no

- ha visto variabilidad de tipo, ubicación, intensidad o duración de interrupciones.
3. **Capacidad constante para toda la red** (35000 pasajeros/hora): no diferencia tramos troncales de alta capacidad (Línea 1, 2, 3) de tramos periféricos.
 4. **Horizonte de predicción discretizado a 1 hora**, mientras el objetivo de la tesis pide horizonte de 10–60 min. Falta granularidad sub-horaria.
 5. **Perfiles origen/destino hardcodedos** (listas fijas de 10 estaciones): no se derivan de un análisis estadístico de la matriz de afluencia real, sino de un supuesto manual razonado.

Ninguno de estos puntos invalida el trabajo — son exactamente el tipo de simplificación esperable en una primera iteración de prototipo — pero deben documentarse como alcance y quedar como candidatos directos para la siguiente fase.

Siguiente paso

Prompt listo para usar en la siguiente sesión de trabajo:

"Extiende `simulador_congestion.py` para generar múltiples días de simulación (mínimo 7–14 días sintéticos, variando entre perfil laboral y fin de semana) en lugar de un solo día fijo (`2026-01-13`). Refactoriza `generador_sintetico_horario.py` para parametrizar el día de simulación y producir una matriz O-D por cada día generado. Luego, sustituye la inyección de interrupción determinista (falla fija de Línea 9 entre 7–9am) por un **generador de eventos estocásticos** parametrizado por tipo de evento (lluvia, falla mecánica, incidente de plataforma), con probabilidad de ocurrencia por hora/línea/temporada (proceso de Poisson u otra distribución de conteo) y magnitud/duración muestreadas de una distribución realista. Añade columnas de contexto al dataset de entrenamiento (`hay_evento` , `tipo_evento` , `severidad_evento`) para que `entrenador_anticipatorio.py` pueda aprender la relación entre eventos estocásticos y congestión futura, y reentrena el modelo con el dataset ampliado. Reporta el cambio en MAE/RMSE frente a la versión actual de un solo día sin eventos aleatorios."

Análisis de factibilidad técnica: eventos estocásticos (lluvia, contingencias/accidentes)

Conclusión general: es técnicamente factible y compatible con la arquitectura

existente, con esfuerzo moderado. El diseño actual ya separa la "capa de infraestructura" (grafo estático), la "capa de demanda" (matriz O-D) y la "capa de estrés" (función BPR + inyección de carga), lo que da puntos de extensión naturales para insertar estocasticidad sin rediseñar el sistema desde cero.

3.1 Dónde se integra cada tipo de evento

Evento	Efecto físico a modelar	Punto de integración en el código actual
Lluvia	Reduce velocidad de acceso a estaciones de superficie/elevadas (Línea A, B, 12 en tramos), incrementa afluencia por trasvase modal (gente que deja camión/bici por metro)	Multiplicador probabilístico sobre <code>peso_atractividad / pasajeros_viaje</code> en <code>generador_sintetico_horario.py</code> , y/o reducción de <code>CAPACIDAD_PROMEDIO_TRAMO_HORA</code> en tramos de superficie dentro de <code>simulador_congestion.py</code>
Falla mecánica / contingencia de línea	Reduce o anula capacidad de un tramo específico por tiempo variable	Ya existe el mecanismo (inyección de carga fantasma en <code>simulador_congestion.py</code>); solo falta parametrizarlo estocásticamente en vez de hardcodearlo
Accidente/incidente de plataforma	Cierre temporal de estación/tramo (grafo con disponibilidad reducida, no solo más lento)	Requiere nueva función que remueva temporalmente una arista o nodo del grafo (<code>G_hora.remove_edge(...)</code>) en vez de solo penalizar el peso – el grafo actual no maneja indisponibilidad total, solo congestión

3.2 Requisitos técnicos adicionales

- 1. Generador de eventos por proceso estocástico.** Un proceso de Poisson (o Binomial Negativa si se quiere sobredispersión) por hora/línea es suficiente y barato computacionalmente: para cada hora simulada, se sortea si ocurre un evento y, si ocurre, se sortea tipo, tramo afectado, severidad y duración. Esto es perfectamente compatible con el bucle `for hora in horas_operacion` que ya existe en `simulador_congestion.py`.
- 2. Fuente de probabilidades realistas.** Aquí está el reto principal, no el técnico sino el de datos: no hay acceso (según la justificación de la propia tesis) a un feed en tiempo real de incidentes del STC Metro. Opciones viables:
 - Usar reportes históricos públicos (redes sociales oficiales del Metro, prensa, boletines de contingencia) para estimar frecuencia aproximada de incidentes por

línea — trabajo de recolección manual/scraping, factible en el tiempo de tesis a escala pequeña (decenas de eventos, no miles).

- Alternativamente (y es lo recomendable dado el alcance de "prototipo validado con datos sintéticos" que ya declara el anexo), **parametrizar la probabilidad a mano con justificación teórica/documental** (p. ej. "la Línea A reporta X incidentes por trimestre según nota de prensa Y"), dejando explícito que es una aproximación y no una calibración estadística rigurosa. Esto es consistente con el marco metodológico ya aceptado (datos sintéticos, no datos reales en tiempo real).
- Para lluvia específicamente, sí existe una fuente pública razonable: climatología histórica de la CDMX (SMN/Conagua) para estimar la probabilidad de lluvia fuerte por mes/hora, que es más defendible que inventar una tasa arbitraria.

3. **Grafo con disponibilidad variable.** El código actual (`grafo_base_metro.gexf`, copiado cada hora) soporta perfectamente eliminar/restaurar aristas por snapshot, ya que cada `G_hora = G_base.copy()` es independiente. No se requiere cambio estructural de librería (sigue siendo NetworkX), solo lógica adicional.

4. **Impacto en el modelo predictivo.** Añadir eventos estocásticos exige ampliar las features del `RandomForestRegressor` (o migrar a un modelo que maneje mejor variables categóricas/exógenas, ej. Gradient Boosting con variables dummy de evento). El riesgo real es de **datos, no de modelo**: con solo 3,740 filas de un día, meter eventos raros (baja frecuencia por diseño) puede dejar clases extremadamente desbalanceadas. Esto refuerza la necesidad del paso previo (varios días simulados) antes de sumar estocasticidad, para tener suficientes muestras positivas de "hubo evento" en el set de entrenamiento.

5. **Costo computacional.** Trivial. Simular N días $\times$ 19 horas $\times$ un sorteo de evento por hora es una operación de milisegundos; el cuello de botella sigue siendo el ruteo de la matriz O-D con `nx.shortest_path` por cada par origen-destino, que ya corre hoy sin problema para un día.

3.3 Riesgos y mitigaciones

- **Riesgo:** sobre-ingeniería del generador de eventos distrayendo del objetivo central (ruteo anticipatorio). *Mitigación:* limitar el generador a 2-3 tipos de evento (lluvia, falla de tramo, cierre de estación) con una distribución simple, documentando explícitamente que el objetivo es probar que el sistema **reacciona correctamente ante disponibilidad variable del grafo**, no producir un simulador meteorológico/operativo exhaustivo.
- **Riesgo:** que la probabilidad "inventada" reste rigor metodológico ante el jurado. *Mitigación:* citar fuente para cada parámetro (aunque sea aproximado) y encuadrarlo explícitamente como supuesto de simulación, tal como ya se hace con los perfiles origen/destino y la función BPR actual — el anexo mismo justifica el uso de simulación por la dificultad de acceso a datos reales en tiempo real.

Veredicto: la integración es factible dentro del alcance de la tesis y no requiere nueva infraestructura ni librerías; el trabajo pendiente es principalmente de diseño de parámetros (tasas de ocurrencia) y de expandir el dataset a múltiples días para que el modelo tenga suficientes ejemplos de eventos disruptivos que aprender.